

DLOs-WM 物理条件化数据管线与图建模改造任务书

一、任务目标

在现有 DLOs-WM 项目基础上，构建一套物理条件化、环境感知、可用于跨材料泛化的 DLO 世界模型数据管线。

本阶段不是追求更大的 GNN，也不是立即实现复杂结绳任务，而是确保模型能够明确区分：

1. DLO 当前处于什么几何和运动状态；
2. 这根 DLO 的物理属性是什么；
3. 哪些节点被固定或抓取；
4. 机器人正在施加什么控制；
5. DLO 与自身、环境和夹爪之间存在什么交互关系；
6. 模型是否学到了受材料和环境控制的动力学，而不是记住单一绳子的轨迹模式。

最终需要支持以下研究问题：

在相同动作和初始形状下，模型能否根据不同的质量、半径、拉伸刚度、弯曲刚度、扭转刚度和摩擦参数，预测不同的 DLO 动力学结果？

二、总体设计原则

2.1 将数据严格分成四类

不要把所有变量都塞进单一 `DLOState`。

A. Episode-level material condition

一条轨迹中基本不变的物理条件：

- segment rest length；
- segment radius；
- linear density；
- stretching stiffness K ；
- bending stiffness E ；
- torsional stiffness G ；
- self-contact static friction；
- self-contact kinetic friction。

这些量是世界模型的条件输入，不是模型需要逐帧预测的状态。

B. Frame-level dynamic state

每个时间步都可能变化：

- node position ;
- node velocity ;
- tension 或 strain proxy ;
- fixed mask ;
- grasped mask ;
- obstacle signed distance ;
- nearest obstacle normal ;
- contact state ;
- topology/event label。

C. Action

机器人或控制器在当前时间步施加的控制：

- grasped node index ;
- gripper ID ;
- target position ;
- target velocity ;
- grasp active ;
- 必要时保存 command delta。

D. Graph structure

描述节点之间物理关系的边：

- structural edge ;
- self-contact edge ;
- environment-contact edge ;
- gripper-attachment edge。

2.2 不要试图从仿真轨迹反推已知参数

在仿真阶段：

- K, E, G ;
- 半径 ;
- 质量 ;
- 摩擦 ;
- 固定节点 ;

都是场景生成时主动设置的参数。

因此应当在创建仿真实体时直接保存这些参数，而不是后续根据轨迹估计。

只有未来做真实世界部署时，才需要进行 system identification 或 latent material inference。

2.3 第一阶段采用最小可用方案

本阶段优先实现：

- DLO-only graph；
- 每个 DLO 节点附带 obstacle SDF；
- 显式 edge type；
- episode-level material condition。

暂时不要优先实现：

- 大量障碍物表面节点；
 - 完整异构图；
 - 可微参数辨识；
 - 真实机器人数据；
 - 通用 knot invariant；
 - 复杂动态障碍物；
 - 端到端视觉输入。
-

三、P0：数据结构重构

这是所有后续任务的基础，必须首先完成。

任务 3.1：新增材料条件数据结构

建议在：

```
dlo_wm/data/schema.py
```

中增加：

```
@dataclass
class MaterialCondition:
    rest_length: torch.Tensor
    radius: torch.Tensor
    linear_density: torch.Tensor
```

```
K: torch.Tensor
E: torch.Tensor
G: torch.Tensor

mu_self_static: torch.Tensor
mu_self_kinetic: torch.Tensor
```

建议形状：

```
rest_length:      [N-1]
radius:           [N-1] 或 scalar
linear_density:   [N-1] 或 scalar
K:                scalar 或 [N-1]
E:                scalar 或 [N-1]
G:                scalar 或 [N-1]
mu_self_static:   scalar
mu_self_kinetic:  scalar
```

第一版可以假设整条 DLO 材料均匀，因此 `radius`、`K`、`E`、`G` 可以保存为 scalar。

但在进入 GNN 前，应展开或编码成 latent。

为什么要做

当前模型只能看到位置、速度和接触等结果，却不知道这些结果是在什么物理条件下产生的。

如果两条轨迹：

- 初始形状相同；
- 动作相同；
- 但弯曲刚度不同；

那么下一帧状态可能不同。

如果模型没有材料参数输入，它面对的是一个一对多映射：

$$(s_t, a_t) \rightarrow s_{t+1}^{(1)}$$

或：

$$(s_t, a_t) \rightarrow s_{t+1}^{(2)}$$

这会导致模型只能预测平均结果，或依赖数据集偏差猜测材料类型。

验收标准

- 可以打印任意 episode 的完整材料参数；
- 不同 episode 可以拥有不同材料条件；
- 同一 episode 内材料参数不会随 frame 意外变化；
- 保存和加载数据后，参数值与生成时完全一致。

任务 3.2：扩展 DLOState

建议调整为：

```
@dataclass
class DLOState:
    pos: torch.Tensor
    vel: torch.Tensor

    tension: torch.Tensor
    contact: torch.Tensor

    fixed_mask: torch.Tensor
    grasped_mask: torch.Tensor

    obstacle_sdf: torch.Tensor
    obstacle_normal: torch.Tensor
    nearest_obstacle_id: torch.Tensor

    topology: torch.Tensor | None = None
```

推荐形状：

```
pos:           [N, 3]
vel:           [N, 3]
tension:       [N, 1]
contact:       [N, 1]
fixed_mask:    [N, 1]
grasped_mask:  [N, 1]
obstacle_sdf:  [N, 1]
obstacle_normal: [N, 3]
nearest_obstacle_id: [N]
```

为什么要做

`fixed_mask` 和 `grasped_mask` 是边界条件。

位置完全相同的节点：

- 如果是自由节点，会受重力和内部弹性作用；
- 如果是固定节点，不会运动；
- 如果是被夹爪控制的节点，会遵循控制速度。

如果不给模型这些 mask，模型无法从位置本身判断节点受到哪种约束。

`obstacle_sdf` 和法向则用于告诉模型节点距离障碍物多远、障碍物位于哪个方向。

验收标准

- 固定节点对应 `fixed_mask=1`；
- 当前被抓取节点对应 `grasped_mask=1`；
- 松开抓取后 `grasped_mask` 能恢复为 0；
- 桌面上方节点的 SDF 为正；
- 穿入障碍物内部时 SDF 为负；
- `obstacle_normal` 方向指向障碍物外部。

任务 3.3：扩展动作数据结构

建议实现：

```
@dataclass
class DLOAction:
    grasp_idx: torch.Tensor
    gripper_id: torch.Tensor

    target_pos: torch.Tensor
    target_vel: torch.Tensor

    grasp_active: torch.Tensor
    delta: torch.Tensor | None = None
```

第一版建议支持单夹爪和双夹爪统一格式：

```
grasp_idx:      [G]
gripper_id:     [G]
target_pos:     [G, 3]
target_vel:     [G, 3]
grasp_active:   [G]
```

其中 `G` 是最大夹爪数量。

未使用的夹爪应通过 `grasp_active=0` 屏蔽。

为什么要做

当前如果只保存 `grasp_idx + delta`，动作语义与：

- 仿真步长；
- macro-step 时长；
- 控制频率；

强耦合。

保存 `target_vel` 后，模型可以直接理解当前施加的运动速度。保存 `target_pos` 则有助于处理位置控制器和控制延迟。

验收标准

验证：

$$target_vel = \frac{target_pos - current_grasp_pos}{macro_dt}$$

在固定速度控制模式下，数值误差应接近零。

任务 3.4：定义 Episode 数据结构

建议实现：

```
@dataclass
class DLOEpisode:
    material: MaterialCondition
    obstacle_metadata: dict

    states: list[DLOState]
    actions: list[DLOAction]

    edge_sets: list[dict] | None

    task_name: str
    seed: int
    episode_id: str
```

为什么要做

材料和障碍物属于整条轨迹，不应在每帧重复保存大量相同数据。

同时，未来按任务、材料或随机种子划分训练集时，需要可靠的 metadata。

验收标准

每条 episode 必须能独立回答：

- 使用了什么材料参数；
- 有哪些障碍物；
- 使用了什么随机种子；
- 属于什么任务；
- 总共有多少帧；
- 每帧使用了什么动作。

四、P0：仿真参数读取与保存

任务 4.1：正确保存 rest length

在创建 rope 并完成 scene build 后、施加重力和动作前，读取：

```
state_initial = rope.get_state()
rest_length = state_initial.length[0].clone()
```

不要使用：

```
rest_length = torch.full(..., interval)
```

也不要再在重力沉降后读取自然长度。

为什么要做

`interval` 是初始化配置，不一定等于 solver 最终使用的自然长度。

如果在重力或抓取后读取，得到的是已经发生形变的当前长度，而不是自然长度。

模型需要通过自然长度计算应变：

$$\epsilon_i = \frac{l_i - l_i^0}{l_i^0}$$

如果自然长度错误，则张力、应变和材料响应都会错误。

验收标准

无外力初始状态下：

$$\frac{|l_i - l_i^0|}{l_i^0}$$

应整体接近零。

任务 4.2：保存半径

从创建 ROD material 时使用的 `segment_radius` 直接保存。

第一版可以：

```
radius = torch.full(
    (num_segments,),
    segment_radius,
    dtype=torch.float32,
)
```

为什么要做

接触不应只由节点中心距离决定。

两段 DLO 是否接触，应考虑：

$$gap_{ij} = distance_{ij} - (r_i + r_j)$$

相同中心距离下，粗线缆可能已经接触，而细绳仍未接触。

验收标准

对人工构造的两段线：

- 距离大于半径和时，不标记接触；
- 距离小于半径和时，标记接触。

任务 4.3：将 segment mass 转换成 linear density

不要只把 `segment_mass` 直接作为模型条件。

计算：

$$\rho_i = \frac{m_i}{l_i^0}$$

实现：

```
linear_density = segment_mass / rest_length.clamp_min(1e-8)
```

为什么要做

`segment_mass` 与离散化分辨率相关。

同一条物理绳：

- 离散成 48 段；
- 离散成 96 段；

每段质量不同，但线密度应相同。

使用线密度更适合跨分辨率泛化。

验收标准

对同一物理 DLO 采用两种离散分辨率时：

- segment mass 不同；
- linear density 应基本一致；
- 总质量应基本一致。

任务 4.4：保存 K、E、G

从场景创建参数中直接保存：

```
material_condition.K = K  
material_condition.E = E  
material_condition.G = G
```

模型输入前建议转换：

```
log_K = torch.log10(K)
log_E = torch.log10(E)
log_G = torch.log10(G)
```

为什么要做

刚度参数可能跨越较大数量级，直接输入原始值会导致：

- 不同输入维度尺度严重不均衡；
- 优化困难；
- 大值特征主导网络；
- 模型难以学习比例变化。

对数变换更接近材料刚度变化的自然尺度。

验收标准

- 保存值必须与创建材料时传入值一致；
- 不允许从 tension proxy 反推出 K, E, G ；
- 归一化后数值不应出现 NaN 或 Inf。

任务 4.5：显式设置并保存摩擦参数

至少区分：

```
mu_self_static
mu_self_kinetic
mu_environment
```

建议在创建 ROD material 时显式传入：

```
static_friction=mu_self_static
kinetic_friction=mu_self_kinetic
```

障碍物 metadata 中保存各自摩擦：

```
{
  "obstacle_id": 1,
  "type": "cylinder",
```

```
"friction": 0.3,  
}
```

为什么要做

DLO 自摩擦和 DLO—环境摩擦控制不同物理现象：

- 自摩擦影响打环后是否容易滑动；
- 桌面摩擦影响拖拽；
- 柱体摩擦影响绕柱状态是否稳定；
- 夹爪摩擦影响是否滑脱。

使用单个 friction scalar 会混淆这些作用。

验收标准

随机改变桌面摩擦时：

- 数据 metadata 正确变化；
- 自摩擦保持不变；
- 两者不会被错误覆盖为同一个值。

五、P0：固定与抓取状态

任务 5.1：生成 fixed mask

根据 `anchor_ids` 创建：

```
fixed_mask = torch.zeros(num_nodes, 1)  
fixed_mask[anchor_ids] = 1.0
```

为什么要做

固定节点是物理边界条件，不能依赖网络自行从轨迹猜测。

否则模型可能错误预测固定点运动，并导致长期 rollout 整体漂移。

验收标准

- fixed 节点在仿真轨迹中的位移应接近零；
- fixed mask 与 anchor ID 一致；
- 如果 episode 中没有固定点，mask 应全部为 0。

任务 5.2：生成 grasped mask

根据当前动作创建：

```
grasped_mask = torch.zeros(num_nodes, 1)
grasped_mask[action.grasp_idx[action.grasp_active]] = 1.0
```

为什么要做

抓取是动态约束：

- 节点可能被抓住；
- 可能被松开；
- 可能切换抓取点；
- 可能同时被两个夹爪控制。

如果只把 action drive 加在节点上，而不显式保存抓取状态，模型难以区分持续抓取与瞬时作用。

验收标准

构造以下单元测试：

1. 抓取节点 10：只有节点 10 的 mask 为 1；
2. 松开：所有节点为 0；
3. 双夹爪抓取节点 5 和 30：两个节点为 1；
4. 抓取点切换后，旧节点恢复为 0。

六、P0：障碍物 SDF

任务 6.1：为基础障碍物实现解析 SDF

第一阶段至少支持：

- plane；
- sphere；
- cylinder；
- box。

实现统一接口：

```
def query_obstacle_sdf(
    points: torch.Tensor,
    obstacle_metadata: list[dict],
) -> tuple[
    torch.Tensor,
    torch.Tensor,
    torch.Tensor,
]:
    """
    Returns:
        nearest_sdf: [N,1]
        nearest_normal: [N,3]
        nearest_obstacle_id: [N]
    """
```

为什么要做

DLO 与环境交互不仅发生在真正碰撞后。

节点接近桌面或柱体时，未来运动已经受到限制。SDF 可以连续表达：

- 距离障碍物多远；
- 是否已经穿入；
- 障碍物在哪个方向。

相比只使用二值 contact，SDF 提供更平滑的学习信号。

验收标准

针对每种解析形状写测试：

- 已知表面点：SDF 接近 0；
- 障碍物外部点：SDF 为正；
- 障碍物内部点：SDF 为负；
- normal 单位长度接近 1；
- normal 指向物体外部。

任务 6.2：保存最近障碍物信息

每个节点至少保存：

```
nearest_sdf
nearest_normal
```

```
nearest_obstacle_id
nearest_obstacle_friction
```

第一版只保存最近障碍物，而不是保存所有障碍物的 SDF。

为什么要做

大部分局部碰撞动力学主要由最近表面决定。

只保存最近障碍物可以控制输入维度，也避免障碍物数量变化导致固定维度困难。

验收标准

在同时存在桌面和柱体时：

- 靠近桌面的节点应选择桌面；
- 靠近柱体的节点应选择柱体；
- `nearest_obstacle_id` 与几何位置一致。

七、P0：边类型重构

任务 7.1：定义 EdgeType

建议定义枚举：

```
class EdgeType(IntEnum):
    STRUCTURAL = 0
    SELF_CONTACT = 1
    ENV_CONTACT = 2
    GRIPPER_ATTACHMENT = 3
```

边特征中使用 one-hot 或 embedding，不再只使用 `is_contact`。

为什么要做

以下关系虽然都可以表示为图边，但物理意义完全不同：

- 结构边负责传递弹性力；
- 自接触边负责碰撞与摩擦；
- 环境接触边连接 DLO 与外部边界；
- 夹爪边表示主动控制约束。

如果只使用 `is_contact`，网络无法区分自接触与环境接触，也无法使用不同的消息传递机制。

验收标准

任意图中的每条边都必须有且只有一个 edge type。

任务 7.2：结构边特征

结构边至少包含：

<code>relative_position</code>	3
<code>current_length</code>	1
<code>rest_length</code>	1
<code>strain</code>	1
<code>radius</code>	1
<code>linear_density</code>	1
<code>edge_type</code>	embedding/one-hot

其中：

$$strain = \frac{current_length - rest_length}{rest_length}$$

为什么要做

仅有当前距离无法判断当前边是否被拉伸。

长度为 2 cm 的边：

- 自然长度为 2 cm 时没有应变；
- 自然长度为 1 cm 时已经严重拉伸。

验收标准

在无外力状态下，平均 strain 接近零；拉伸动作后，相关结构边 strain 上升。

任务 7.3：自接触边

临时版本可以继续使用非相邻节点距离，但应改成半径感知：

$$distance(i, j) < r_i + r_j + \epsilon$$

必须排除：

- 同一节点；
- 相邻结构节点；
- 距离过近但属于同一局部弯曲段的节点。

后续版本再升级为 segment-to-segment 最短距离。

为什么要做

仅使用固定 `contact_radius` 会造成不同半径 DLO 的接触定义不一致。

同时，节点到节点接触会漏掉两条线段中部发生的接触，因此只适合作为第一版。

验收标准

- 相邻结构节点不应被标记为 self-contact；
- 非相邻段接近时产生 self-contact edge；
- 改变 radius 后接触阈值相应变化。

任务 7.4：环境接触边

根据 SDF 建立：

$$contact_{i,o} = 1 [SDF_o(x_i) < r_i + \epsilon]$$

第一阶段不一定需要真实 obstacle node。

可以为每个 DLO 节点增加环境接触特征，或者创建指向虚拟环境节点的边。

为什么要做

自接触与环境接触对应不同摩擦、几何和动力学约束，必须分开。

验收标准

- 节点靠近桌面时产生 env-contact；
- 节点靠近自身另一段绳时产生 self-contact；
- 两种边类型不能混淆。

任务 7.5：夹爪 attachment edge

对于每个 active gripper，可以创建虚拟 gripper node，并连接被抓节点。

边特征可包括：

```
relative target position
target velocity
gripper ID
attachment stiffness
grasp active
```

如果本阶段不想增加虚拟节点，则至少将 action 特征散射到被抓节点，并保留 `grasped_mask`。

为什么要做

把控制信息作为显式图关系，比简单把一个速度向量拼到节点特征中更容易扩展到：

- 双夹爪；
- 换抓点；
- 多接触控制；
- 真实机器人末端状态。

验收标准

第一阶段允许采用 action scatter，但代码接口需要为后续 gripper node 保留扩展空间。

八、P0：数据生成与随机化

任务 8.1：实现 episode-level 参数采样器

新增：

```
def sample_material_condition(
    base_config,
    generator,
) -> dict:
    ...
```

第一阶段建议采用温和随机化：

```
K:          base × [0.5, 2.0]
E:          base × [0.5, 2.0]
G:          base × [0.5, 2.0]
linear density: base × [0.7, 1.5]
```

```
radius:          base × [0.8, 1.2]
self friction:   合理范围均匀采样
env friction:    合理范围均匀采样
```

刚度建议在 log-space 采样：

```
scale = 10 ** uniform(-0.3, 0.3)
```

为什么要做

如果所有 episode 物理参数相同，即使保存了参数，网络也可以完全忽略它们。

只有当同一动作在不同物理条件下产生不同结果时，材料条件才具有监督价值。

验收标准

生成至少 100 个 episode 后，打印参数分布：

- 每个参数必须具有明显方差；
- 参数之间不应全部完全相关；
- 不得出现 solver 不稳定、NaN 或明显爆炸。

任务 8.2：每次改变材料参数时重建场景

不要在轨迹中途直接修改 K , E , G 。

每条 episode：

1. 采样材料参数；
2. 创建新的 material；
3. 创建新的 rope entity；
4. build scene；
5. 读取自然长度；
6. 开始 rollout。

为什么要做

多数物理仿真器会在 entity 构建阶段预计算：

- 质量矩阵；
- 内部约束；
- rest configuration；
- solver 参数。

中途修改 Python 属性未必真正更新底层求解器。

验收标准

检查不同 episode 的底层 material 参数确实不同，而不仅是 metadata 文件不同。

任务 8.3：保存完整版本化数据

推荐格式：

```
torch.save({
    "format_version": 2,
    "episodes": episodes,
    "generator_config": config,
    "normalization": stats,
    "split": split_name,
    "created_at": timestamp,
    "git_commit": commit_hash,
}, output_path)
```

为什么要做

数据 schema 改动后，旧缓存很容易被新代码误读。

加入 `format_version` 后，loader 可以明确拒绝不兼容数据，而不是静默产生错误。

验收标准

loader 必须：

- 支持 version 2；
 - 对未知版本抛出明确错误；
 - 不允许默认假设缺失字段为正确值。
-

九、P0：数据集划分

任务 9.1：按 episode 划分

生成：

```
train.pt
val.pt
test_id.pt
test_ood_material.pt
test_ood_geometry.pt
test_ood_action.pt
```

禁止按照 transition 随机打散后划分。

为什么要做

同一条轨迹的相邻帧高度相似。

如果将相邻 transition 分到训练集和测试集，测试结果会严重虚高，不能反映长期预测或泛化能力。

验收标准

任意一个 `episode_id` 只能出现在一个 split 中。

任务 9.2：定义 OOD material split

例如：

```
训练：
K/E/G scale ∈ [0.7, 1.4]

OOD：
K/E/G scale ∈ [0.5, 0.65] ∪ [1.5, 2.0]
```

也可以将部分参数组合保留为 OOD：

```
训练中出现：
软且轻
硬且重

测试中保留：
软且重
硬且轻
```

为什么要做

普通随机划分只能测试插值。

论文真正需要回答的是：

模型能否根据显式材料条件，对没有在训练中出现过的物理属性组合进行预测？

验收标准

训练和 OOD material 的参数分布应有清晰差异，并生成可视化直方图。

十、P1：模型改造

任务 10.1：新增 Material Encoder

实现：

```
material_latent = material_encoder(  
    [  
        log_K,  
        log_E,  
        log_G,  
        radius,  
        linear_density,  
        mu_self_static,  
        mu_self_kinetic,  
    ]  
)
```

第一版将 latent 广播到所有节点：

```
material_per_node = material_latent.expand(num_nodes, -1)  
node_input = torch.cat(  
    [node_features, material_per_node],  
    dim=-1,  
)
```

为什么要做

仅保存物理参数但不输入模型没有意义。

通过 material encoder，可以将不同量纲和尺度的参数映射到统一 latent 空间。

验收标准

进行材料条件打乱测试：

- 正确材料条件下预测误差较低；
- 将 batch 内材料条件随机打乱后，预测误差应明显增加。

如果打乱后结果完全不变，说明模型没有使用材料条件。

任务 10.2：每层 message passing 注入 material latent

更推荐在每个 processor block 中使用：

```
message = edge_mlp(  
    sender_latent,  
    receiver_latent,  
    edge_latent,  
    material_latent,  
)
```

为什么要做

材料属性影响的是力传播规律，而不仅是初始节点编码。

只在输入层拼接材料 latent，经过多层网络后可能被逐渐稀释。

验收标准

对比：

1. 无 material；
2. 只在 input 注入；
3. 每层注入。

记录 ID 与 OOD rollout 结果。

任务 10.3：edge type embedding

实现：

```
edge_type_latent = nn.Embedding(  
    num_embeddings=4,
```

```
        embedding_dim=edge_type_dim,  
    )
```

不要把 `edge_type` 当成连续 scalar 直接输入。

为什么要做

类型编号 0、1、2、3 不存在数值顺序关系。

Embedding 可以让不同物理关系具有不同的消息传递表示。

验收标准

删除 edge type 后，接触场景预测性能应下降；如果完全无差异，需要检查边类型是否正确进入网络。

任务 10.4：节点输入规范化

分别统计并保存：

```
position mean/std  
velocity mean/std  
target velocity mean/std  
SDF mean/std  
tension/strain mean/std  
log material parameter mean/std
```

mask 和 edge type 不做标准化。

为什么要做

位置、速度、刚度和 SDF 的数值尺度差异可能达到多个数量级。

不规范化容易造成训练不稳定或模型忽略小尺度特征。

验收标准

标准化后的连续变量：

- 均值接近 0；
- 标准差接近 1；
- 无 NaN 和 Inf；
- 验证与测试集使用训练集统计量，不能分别重新计算。

十一、P1：训练任务与损失

任务 11.1：先保持单步预测任务稳定

第一阶段预测：

```
delta position
next velocity
contact
```

暂时不要同时大幅修改：

- 模型结构；
- 多步损失；
- topology 标签；
- failure head；
- planner。

为什么要做

同时改变过多模块会导致结果下降时无法定位原因。

先验证新的数据 schema 与物理条件确实有效，再升级长期 rollout。

验收标准

在固定材料条件下，新模型单步结果至少不应明显差于原模型。

任务 11.2：增加物理一致性辅助损失

可以加入：

结构长度误差

$$L_{length} = \frac{1}{N-1} \sum_i ||\hat{x}_{i+1} - \hat{x}_i|| - l_{i,t+1}|$$

固定点约束

$$L_{fixed} = \sum_i fixed_i ||\hat{x}_{i,t+1} - x_{i,t}||^2$$

抓取点控制一致性

$$L_{grasp} = \sum_i grasped_i \|\hat{x}_{i,t+1} - x_{i,t+1}^{target}\|^2$$

为什么要做

世界模型不能只追求平均位置误差，还应满足明确边界条件。

尤其固定节点发生微小误差后，在长期 rollout 中会造成整体漂移。

验收标准

增加损失后：

- fixed node drift 明显下降；
- edge length violation 不恶化；
- 不允许通过过高辅助损失权重牺牲整体动力学预测。

任务 11.3：实现真正的多步 rollout loss

在单步版本稳定后，实现：

```
pred_state = initial_state

for k in range(horizon):
    pred_state = model(
        pred_state,
        actions[k],
        material,
        environment,
    )
    loss += weight[k] * state_loss(
        pred_state,
        gt_states[k + 1],
    )
```

训练 curriculum：

```
H=1
H=2
H=4
```

H=8
H=16

为什么要做

仅在输入状态上加噪声，不等同于让模型面对自己的预测误差。

真正的世界模型需要在模型自己生成的状态分布上继续预测，否则长期 rollout 会快速偏离。

验收标准

报告：

RMSE@1
RMSE@5
RMSE@10
RMSE@20

并确保多步训练相比纯单步训练在长 horizon 上有明显优势。

十二、P1：评估实验

任务 12.1：建立核心 baseline

至少比较：

1. 原始 GNN；
2. 原始 GNN + 新状态特征，但无 material condition；
3. material-conditioned GNN；
4. material-conditioned GNN + edge type；
5. material-conditioned GNN + edge type + SDF。

为什么要做

必须证明提升分别来自：

- 物理条件；
- 接触关系；
- 环境信息；

而不是模型参数量增加。

验收标准

所有 baseline：

- 使用相同数据；
- 使用相同训练步数；
- 使用相同 hidden size 或报告参数量；
- 至少运行 3 个随机种子。

任务 12.2：材料条件打乱实验

测试时将材料 condition 在 batch 内随机重排。

比较：

正确条件
错误条件
无条件

为什么要做

这是验证网络是否真正使用材料参数的直接实验。

如果使用错误材料条件仍得到相同结果，模型可能只是依靠当前几何和速度预测，材料 encoder 没有实际作用。

验收标准

在不同材料产生明显不同动力学的测试集上，错误条件应显著降低预测性能。

任务 12.3：反事实预测实验

选择相同：

- 初始状态；
- 动作序列；
- 障碍物；

只改变 E 、质量或摩擦。

分别让 simulator 和 world model rollout。

为什么要做

这是最有解释性的定性实验之一。

它可以展示模型是否学会：

- 软绳弯曲更多；
- 重绳下垂更多；
- 高摩擦情况下滑动更少；
- 高扭转刚度情况下扭曲传播不同。

验收标准

模型预测的变化方向应与 simulator 一致，并展示轨迹可视化。

任务 12.4：报告物理与图指标

至少包括：

```
position RMSE @ 1/5/10/20
velocity RMSE
Chamfer distance
edge-length violation
fixed-node drift
self-contact precision/recall/F1
environment-contact precision/recall/F1
rollout divergence rate
```

为什么要做

单纯位置误差无法衡量：

- 绳子是否被错误拉长；
- 接触关系是否错误；
- 固定点是否漂移；
- 拓扑关系是否破坏。

验收标准

评估脚本应生成统一 JSON 或 CSV 文件，方便后续绘图和论文表格生成。

十三、P1：测试与调试

任务 13.1：为 schema 添加 round-trip test

测试流程：

1. 创建人工 episode；
2. 保存到磁盘；
3. 重新加载；
4. 比较所有 tensor；
5. 检查 dtype、shape 和 device。

为什么要做

数据格式一旦错误，会污染所有训练结果，而且通常很难在 loss 中直接发现。

任务 13.2：物理参数敏感性测试

构造相同初始状态和动作，只改变一个参数：

低 E vs 高 E
低质量 vs 高质量
低摩擦 vs 高摩擦

检查 simulator 结果是否确实存在差异。

为什么要做

如果随机化参数对仿真结果几乎没有影响，那么保存这些参数也无法为模型提供有效监督。

也可能说明：

- 参数范围太窄；
- 参数没有正确传入底层 solver；
- 场景动作不足以激发对应物理属性。

验收标准

每种参数至少设计一个能明显激发其作用的动作：

- K ：轴向拉伸；
- E ：侧向弯曲；

- G ：端部旋转；
 - 质量：悬垂；
 - 摩擦：桌面拖拽或绕柱滑动。
-

任务 13.3：设备一致性测试

所有动态构图代码必须使用：

```
device = pos.device
```

禁止出现隐式 CPU tensor，例如：

```
torch.arange(n)
```

应改为：

```
torch.arange(n, device=pos.device)
```

为什么要做

CPU/GPU device mismatch 常在训练正常、rollout 才出现，尤其是动态构图阶段。

验收标准

完整执行：

- CPU 数据生成；
 - 单 GPU 训练；
 - GPU rollout；
 - batch graph；
 - 多 GPU/DDP 冒烟测试。
-

十四、暂时不要做的事项

在完成上述 P0/P1 前，不要优先投入以下内容：

- 通用 knot classification；
- 使用 crossing count 作为主要 topology contribution；
- 更大的 Transformer；

- 图像端到端输入；
- 大规模真实数据采集；
- 复杂双臂机器人控制器；
- diffusion world model；
- latent action model；
- uncertainty ensemble；
- 完整 CEM/MPPI planner；
- 多物体异构场景；
- 从真实视频反推 K, E, G 。

原因是当前首要风险不是模型容量不足，而是：

- 数据条件不完整；
- 物理参数没有进入模型；
- 边类型混合；
- 评估划分不可信；
- 长期 rollout 训练尚未完善。

十五、建议实施顺序

阶段 A：数据 schema 和仿真条件

按顺序完成：

- ☐ MaterialCondition
- ☐ 扩展 DLOState
- ☐ 扩展 DLOAction
- ☐ 新增 DLOEpisode
- ☐ 读取真实 rest length
- ☐ 保存 radius 和 linear density
- ☐ 保存 K, E, G
- ☐ 显式设置并保存摩擦
- ☐ fixed/grasped mask
- ☐ 数据版本号
- ☐ round-trip test

完成标准：能够生成和可靠加载单条完整 episode。

阶段 B：环境和图结构

- ☐ 实现 plane SDF
- ☐ 实现 sphere/cylinder/box SDF
- ☐ 保存 SDF、normal 和 obstacle ID

- ☐ 定义 EdgeType
- ☐ 重构 structural edge
- ☐ 半径感知 self-contact
- ☐ environment-contact
- ☐ action scatter 或 gripper attachment

完成标准：可视化任意帧时，能够清楚展示四类关系和节点环境特征。

阶段 C：材料随机化与数据集

- ☐ 实现 episode-level 参数采样
- ☐ 每条 episode 重建 scene
- ☐ 生成至少 100 条冒烟数据
- ☐ 检查参数分布
- ☐ 检查参数是否真实影响仿真
- ☐ 生成正式训练数据
- ☐ 建立 trajectory-level split
- ☐ 建立 OOD material split

完成标准：同一动作在不同材料条件下产生可观察的差异。

阶段 D：模型

- ☐ Material Encoder
- ☐ material latent 广播到节点
- ☐ 每层 message passing 注入材料条件
- ☐ edge type embedding
- ☐ SDF 和法向进入 node encoder
- ☐ 输入归一化
- ☐ 保持原单步任务可训练

完成标准：正确条件优于错误或无条件模型。

阶段 E：多步预测与实验

- ☐ fixed-point loss
- ☐ edge-length loss
- ☐ grasp consistency loss
- ☐ 真正的 autoregressive rollout loss
- ☐ horizon curriculum
- ☐ baseline 和消融
- ☐ 材料打乱实验
- ☐ 反事实材料实验

- [] ID/OOD 长期 rollout
- [] 输出统一评估文件

完成标准：material-conditioned 模型在 OOD material 和长期 rollout 上获得稳定提升。

十六、建议提交拆分

建议不要一次提交全部改动，而是拆成以下 commits：

1. refactor: introduce versioned episode and material schemas
2. feat: export physical material parameters from DLO-Lab
3. feat: add fixed and grasped node masks
4. feat: add analytic obstacle SDF queries
5. refactor: introduce typed graph edges
6. feat: add episode-level physics randomization
7. feat: create trajectory-level ID and OOD splits
8. feat: add material-conditioned GNN encoder
9. feat: inject material context into message passing
10. feat: add physics-aware losses
11. feat: implement autoregressive rollout training
12. eval: add material OOD and counterfactual benchmarks

每个 commit 必须：

- 能运行基础测试；
 - 不提交大型数据文件；
 - 不混合无关格式化；
 - 包含必要的配置更新；
 - 在 commit message 或 PR 描述中解释 schema 是否发生不兼容变化。
-

十七、最终交付物

智能体完成后应提供：

代码

- 更新后的 schema；
- 更新后的 DLO-Lab provider；
- 参数随机化器；
- SDF 模块；
- typed edge builder；

- versioned dataset loader ;
- material-conditioned GNN ;
- 多步 rollout trainer ;
- 评估脚本 ;
- 单元测试。

配置

至少提供：

```
config/data/default_physics.yaml
config/data/material_randomization.yaml
config/data/ood_material.yaml
config/model/material_conditioned_gnn.yaml
config/train/rollout_curriculum.yaml
```

文档

新增或更新 README，说明：

- 每个数据字段的物理意义；
- 单位；
- tensor shape；
- 数据生成命令；
- 训练命令；
- 评估命令；
- ID/OOD split 规则；
- 已知限制。

实验输出

至少生成：

- 单步 ID 结果；
- 多步 ID 结果；
- OOD material 结果；
- material shuffle 结果；
- 三个反事实可视化；
- contact F1；
- edge-length violation；
- fixed-node drift；
- 参数分布图。

十八、最终验收清单

只有全部满足，才能认为本阶段完成：

- [] 数据中包含真实 rest length；
- [] 使用 linear density，而不是只使用 segment mass；
- [] K, E, G 是场景生成条件，不是从结果反推；
- [] 自摩擦与环境摩擦分开；
- [] fixed mask 与 grasped mask 分开；
- [] action 包含 target position 和 target velocity；
- [] 每节点具有 SDF 和 obstacle normal；
- [] structural/self-contact/environment-contact 不再混为同一边类型；
- [] 数据按完整 episode 划分；
- [] 有明确 OOD material 测试；
- [] 模型能够接收 material condition；
- [] material condition 打乱后结果明显恶化；
- [] 多步训练使用模型自己的预测状态；
- [] 所有连续输入使用训练集统计量归一化；
- [] CPU、GPU 和 batch graph 测试均通过；
- [] 数据文件具有版本号；
- [] 代码中没有静默填补缺失物理字段；
- [] 实验能证明模型学习的是条件动力学，而非单一数据分布记忆。

十九、给执行智能体的最高优先级要求

1. 先读现有实现，再修改。不要根据任务书假设当前函数签名。
2. 保持向后兼容不是首要目标。如旧 schema 不合理，可以升级版本并显式拒绝旧数据。
3. 不要伪造仿真器没有提供的 **ground truth**。无法直接读取的量必须明确标记为 geometry proxy。
4. 每增加一个字段，都要说明来源、单位、**shape** 和生命周期。
5. 每完成一阶段，都先运行最小测试，不要等全部开发结束再调试。
6. 不要用增大模型容量掩盖数据定义问题。
7. 任何实验提升都必须有对应消融。
8. 发现 DLO-Lab API 与任务书描述不同时，以本地实际版本为准，并在报告中记录差异。
9. 不要把 **crossing count** 宣称为严格拓扑不变量。
10. 最终输出应包括修改文件清单、运行命令、测试结果和未完成事项。